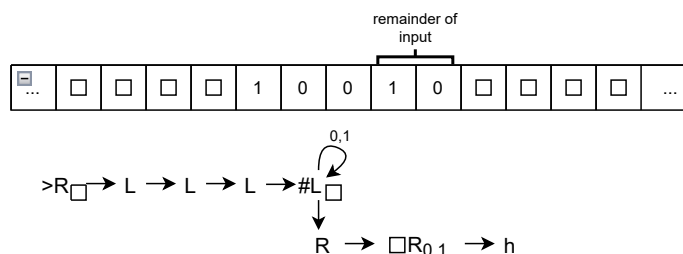


1. A deterministic Turing machine M that, given as input a binary string w , computes the remainder of w modulo 4. Where M starts with the initial configuration (s, w) and halts with the configuration $(h, (w \bmod 4)2)$. Assuming that the input, w , is a valid nonnegative number in base 2, that is, $w \in \{0\} \cup \{0, 1\}^*$ the Turing machine M would need to:

- Go right until a blank space is identified
- Go left 3 spaces to be in the 4 binary placement
- Mark off the 4 placement with a symbol (#)
- Then go left until another blank space is identified
- With each step left, replace the digit with the same symbol (#). All of these binary digits to the left of the 4 binary placement would all be divisible by 4 so they would not need to be part of the calculation for the remainder of the input (mod 4).
- Once the blank space is reached, the Turing machine would then go all the way to right, replacing all # symbols blank space symbols,
- The machine will halt once it reaches the binary digits leftover
- These leftover digits will make the remainder of the input (mod 4)



2a.) $L = \{ \langle M \rangle \mid M \text{ has 3 states} \}$

A Turing machine M_a can be constructed to decide the language L . This Turing machine M , would work by simulating through the machine $\langle M \rangle$, counting the states as it goes along. It would go through 3 states and if there is nothing else, then accept. But, if there are more than 3 states or the machine halts after iterating through less than 3 states, then reject. This Turing machine M would make L decidable.

2b.) $L = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \in \text{SD} \}$

Using the theorem stating that a language is in SD if and only if it is Turing-enumerable and vice versa. A Turing machine M can be constructed. If L as a subset of Σ^* is in SD, then there is a Turing machine that semi-decides L . This means that the language of the TM that has its acceptable language in SD would always have input that is either accepted or rejected (part of the semi-decidable Turing machine or not). Therefore a Turing machine M_b can be constructed for L , making L decidable.

2c.) $L = \{ \langle M, w \rangle \mid M \text{ halts on the input } w \text{ in at most } 3|w| \text{ steps} \}$

A Turing machine M_c can be constructed to decide the language L . This Turing machine would go through and simulate M for $3|w|$ steps. If M halts at any point during the simulation, the Turing machine M_c would accept. If the machine continues past $3|w|$ steps then M_c would reject. Therefore, because a Turing machine can be constructed for L , then L is decidable.

2d.) $L = \{ \langle M, w \rangle \mid M \text{ uses only at most } |w| \text{ tape cells before and after the input } w \text{ (maximum } 3|w| \text{ tape cells in total)} \}$

A Turing machine M_d can be constructed to decide L . This Turing machine M would start at the head of the tape and run through it. If it encounters a blank space before it reads $3|w|$ cells then the Turing machine M_d would accept. If no blank space is encountered before then and $3|w|+1$ cells are read from the tape, then M_d would reject. Because a Turing machine can be made to decide L , then L is decidable.

3.) A Turing machine is enumerable if and only if a language is semi-decidable (in SD) and vice versa. If and only if, for some fixed, non-halting state of the Turing machines $(M, M_1, M_2, \dots, M_n)$ the answer would be yes, a Turing machine can be constructed so that it can enumerate the encoding of all other Turing machines that have non empty languages. Excluding specific cases of simple Turing machines that have no transitions, a Turing machine can take its input as encoding of another Turing machine $(\langle M_1 \rangle)$ or even its own encoding, performing some operation on it, and that Turing machine would be able to perform that operation on an empty input tape, outputting a list of that Turing machines possible outputs. However, if the encoding for a Turing machines is not non-halting (looping), then this Turing machine would not be able to halt (decidable). Therefore, making the language of all Turing machine encodings semi-decidable. So for all syntactically valid Turing machines whose input alphabet is some particular set of Σ and whose tape alphabet is some particular set Γ there exists an infinite lexicographical enumeration.

4.) Language L not being decidable ($L \notin D$) implies that there is not a Turing machine that can define L also meaning that L is either semi-decidable (LD) implying that its complement would also be semi-decidable, or it could mean that both L and its complement are not semi-decidable. The context-free language C would imply that C is also both decidable ($C \in D$) and semi-decidable ($C \in \text{SD}$) because context-free languages are a proper subset of Decidable languages and Decidable languages are a proper subset of Undecidable Languages. This means that there exists a context-free grammar and a Turing machine that can generate and decide it. This would mean that the union of L and C would be a set within the undecidable language range overall, where there is not a Turing machine that can decide all possible inputs, therefore it is undecidable. Some of the inputs can be accepted, by both L and C individually and their Turing machine will halt. Some, such as the instance when C is an empty set or if the input is semi-decidable, may not accept leading to the union of C and L being semi-decidable (able to accept some inputs and not accept others).

5.) Because SD cannot be closed under complement due to its potential to never reach a halting state, meaning that SD is not a subset of the closure of SD, giving a class of languages (the closure of SD) that is strictly larger than SD. The Church-Turing thesis claims that all formalisms powerful enough to describe everything we think of as a computational algorithm are equivalent. Therefore the original statement would not contradict the Church-Turing thesis, because the set of SDc would lie outside of the scope we deem "computable" but the size of the set would not impact the power of the computational algorithms dedicated to it.